

Safe and Unsafe States

1 Main Idea

System State

In deadlock avoidance, the current state is described by the resource data structures E , A , C , and R : existing resources, available resources, current allocation, and remaining requests.

A deadlock-avoidance algorithm checks whether granting a resource request keeps the system in a safe state.

Safe State

A state is **safe** if there exists at least one scheduling order in which every process can run to completion, even if all processes immediately request their maximum remaining resources.

Unsafe State

A state is **unsafe** if the system cannot guarantee that all processes can finish. An unsafe state is not necessarily deadlocked, but it may lead to deadlock.

2 Safe Sequence

Definition

A **safe sequence** is an order of processes $P_1, P_2, \dots, P_n$ such that each process can receive its remaining needed resources, finish, and release its resources for the next process.

Test for One Resource Type

For a process P_i ,

$$\text{Need}_i = \text{Max}_i - \text{Has}_i$$

$$P_i \text{ can finish if } \text{Need}_i \leq \text{Free}$$

3 Example 1: A Safe State

Initial State

There are 10 total instances of one resource. Processes A , B , and C currently hold some instances and may need more later.

Process	Has	Max	Need = Max - Has
A	3	9	6
B	2	4	2
C	2	7	5

$$\text{Allocated} = 3 + 2 + 2 = 7, \quad \text{Free} = 10 - 7 = 3$$

Since B 's need is 2, and $2 \leq 3$, process B can safely finish first.

4 Safe-State Trace

Step	Free	Action	New free
(a)	3	Run B . It needs 2 more, so it can reach its max 4.	1 while B runs
(b)	1	B finishes and releases all 4 resources.	$1 + 4 = 5$
(c)	5	Run C . It needs 5 more, so it can reach its max 7.	0 while C runs
(d)	0	C finishes and releases all 7 resources.	$0 + 7 = 7$
(e)	7	Run A . It needs 6 more, so it can finish.	all processes finish

Safe Sequence

The state is safe because the sequence

$$B \rightarrow C \rightarrow A$$

allows every process to finish.

5 Visual Trace of the Safe State

	Has	Max
A	3	9
B	2	4
C	2	7
Free: 3		
(a)		

	Has	Max
A	3	9
B	4	4
C	2	7
Free: 1		
(b)		

	Has	Max
A	3	9
B	0	–
C	2	7
Free: 5		
(c)		

	Has	Max
A	3	9
B	0	–
C	7	7
Free: 0		
(d)		

	Has	Max
A	3	9
B	0	–
C	0	–
Free: 7		
(e)		

Transition	Meaning
(a) $\rightarrow$ (b)	B receives 2 more resources and reaches its maximum of 4.
(b) $\rightarrow$ (c)	B finishes, so its 4 resources return to the free pool.
(c) $\rightarrow$ (d)	C receives 5 more resources and reaches its maximum of 7.
(d) $\rightarrow$ (e)	C finishes, so A can later receive what it needs.

Figure: Safe sequence $B \rightarrow C \rightarrow A$. A dash means the process has finished.

6 Example 2: An Unsafe State

Now suppose process A requests and receives one more resource.

9 Unsafe Does Not Mean Deadlocked

Important distinction

An unsafe state is not the same as a deadlocked state.

Starting from the unsafe state, the system may still run for a while. For example, B can complete. It is also possible that A might release a resource before asking for more, allowing C to finish. However, the system cannot guarantee completion under worst-case requests.

State type	Meaning
Safe	There exists a guaranteed order in which all processes can finish.
Unsafe	No guaranteed completion order exists, but deadlock may not have happened yet.
Deadlocked	Processes are already stuck waiting for events only each other can cause.